

**Práctica No. 2:**  
**Números (pseudo)aleatorios y Generación de Variables**  
**Aleatorias**

Informe de Laboratorio  
Simulación Computacional

**J. Gregorio Delgado**  
Código: 160005010

Universidad de los Llanos  
Facultad de Ciencias Básicas e Ingenierías  
Programa de Ingeniería de Sistemas  
Villavicencio, Meta  
Septiembre de 2026



## 1. Objetivos

La práctica implementa y valida generadores de números pseudoaleatorios y usa sus salidas para producir variables aleatorias con distribuciones dadas. Se cubren el método de cuadrados medios, el congruencial mixto y dos generadores tomados de [2], calculando para cada secuencia el tamaño de ciclo y aplicando una prueba de uniformidad y una de aleatoriedad. Con los uniformes validados se estiman siete integrales por Monte Carlo, se generan variables discretas por transformada inversa y composición, y se repite la simulación de la cola del Laboratorio 1 alimentada por generadores de Poisson y Binomial. El desarrollo completo está en el cuaderno adjunto `Lab2_160005010.ipynb`.

## 2. Consulta previa

### 2.1. ¿Qué es una variable aleatoria?

Es una función  $X : \Omega \rightarrow \mathbb{R}$  que asigna un número real a cada resultado posible de un experimento aleatorio, traduciendo resultados que pueden ser cualitativos a valores numéricos sobre los que sí se puede calcular. Lo aleatorio no está en la función, que es determinista, sino en cuál elemento de  $\Omega$  resulta seleccionado.

### 2.2. ¿Qué es una distribución de probabilidad?

Es la regla que reparte la probabilidad entre los valores que puede tomar una variable aleatoria, exigiendo que sean no negativas y sumen o integren uno sobre todo el soporte. Se describe con la función de masa en el caso discreto, la densidad en el continuo, o la acumulada  $F(x) = P(X \leq x)$ , que sirve para ambos.

### 2.3. Diferencia entre distribución discreta y continua

La distinción está en el soporte. Una variable discreta toma valores en un conjunto numerable, donde cada valor carga probabilidad positiva y la función de masa  $p(x) = P(X = x)$  tiene sentido directo; una continua los toma en un intervalo no numerable, donde la probabilidad de un punto aislado es cero y solo los intervalos tienen probabilidad positiva. Por eso en un caso se suma y en el otro se integra, y la transformada inversa discreta se implementa recorriendo la acumulada en lugar de invertir una función.

### 2.4. Definición de la uniforme continua $U(0, 1)$

Reparte la probabilidad de manera pareja sobre  $[0, 1]$ , con densidad uno dentro del intervalo y cero fuera:

$$f(x) = \begin{cases} 1 & 0 \leq x \leq 1 \\ 0 & \text{en otro caso} \end{cases} \quad F(x) = x \text{ para } 0 \leq x \leq 1 \quad (1)$$

Su media es  $1/2$  y su varianza  $1/12$ . Es la distribución de referencia de toda la simulación, porque los generadores buscan producir muestras que se comporten como si vinieran de ella y a partir de ahí se construye cualquier otra.

### 2.5. Definición de la distribución de Poisson

Modela el número de eventos en un intervalo fijo cuando ocurren de forma independiente a una tasa media constante  $\lambda$ :

$$P(X = i) = \frac{e^{-\lambda} \lambda^i}{i!}, \quad i = 0, 1, 2, \dots \quad (2)$$

Su media y su varianza valen ambas  $\lambda$ , igualdad que en la sección 7 sirve como verificación directa del generador.

### 2.6. Definición de la distribución Binomial

Cuenta los éxitos en  $n$  ensayos independientes con probabilidad  $p$  cada uno:

$$P(X = i) = \binom{n}{i} p^i (1-p)^{n-i}, \quad i = 0, 1, \dots, n \quad (3)$$

Su media es  $np$  y su varianza  $np(1-p)$ ; con  $n = 10$  y  $p = 0,40$  eso da 4 y 2,4 respectivamente.



## 2.7. Propiedad independiente e idénticamente distribuida (i.i.d.)

Una sucesión es i.i.d. cuando todas sus variables siguen la misma distribución y ninguna aporta información sobre las demás. Son condiciones separadas y una no implica la otra, lo que explica por qué la práctica exige dos pruebas: el chi-cuadrado verifica que la secuencia sea idénticamente distribuida según  $U(0,1)$  y el test de rachas verifica la independencia. La sección 4 muestra dos generadores que aprueban la primera y reprueban la segunda.

## 2.8. ¿En qué consiste una prueba estadística?

Es un procedimiento para decidir si los datos observados son compatibles con una hipótesis nula  $H_0$ : se elige un estadístico cuya distribución bajo  $H_0$  se conoce y se compara el valor obtenido contra la región de rechazo que define el nivel de significancia  $\alpha$ . No rechazar no equivale a demostrar, y con  $\alpha = 0,05$  se rechaza por azar una de cada veinte muestras que sí cumplen  $H_0$ , situación que aparece en la sección 4 con MINSTD.

# 3. Generación de números pseudoaleatorios

## 3.1. Método de cuadrados medios

El método eleva la semilla al cuadrado, rellena con ceros a la izquierda hasta  $2d$  dígitos y extrae los  $d$  centrales, que pasan a ser el siguiente valor y la nueva semilla. La cantidad de dígitos se fija con la semilla inicial y se pasa como parámetro, porque recalcularla en cada paso perdería los ceros a la izquierda.

```
1 def obtener_digitos_centrales(numero, cantidad_digitos):
2     cuadrado = numero * numero
3     cuadrado_texto = str(cuadrado).zfill(cantidad_digitos * 2)
4
5     inicio = (len(cuadrado_texto) - cantidad_digitos) // 2
6     return int(cuadrado_texto[inicio:inicio + cantidad_digitos])
```

Listado 1: Extracción de los dígitos centrales.

## 3.2. Generador congruencial mixto

La recurrencia es  $X_{n+1} = (aX_n + c) \bmod m$  y el uniforme resulta de dividir entre  $m$ . Se implementó además la verificación de Hull-Dobell, que determina si el periodo alcanza el máximo  $m$ :  $c$  y  $m$  primos relativos,  $a - 1$  divisible por todo factor primo de  $m$ , y  $a - 1$  múltiplo de 4 cuando  $m$  lo sea.

```
1 def congruencial_mixto(semilla, multiplicador, incremento, modulo, cantidad):
2     actual = semilla
3     enteros, uniformes = [], []
4
5     for _ in range(cantidad):
6         actual = (multiplicador * actual + incremento) % modulo
7         enteros.append(actual)
8         uniformes.append(actual / modulo)
9
10    return enteros, uniformes
```

Listado 2: Núcleo del generador congruencial mixto.

## 3.3. Otros generadores

Se implementaron el shift-register y el de congruencia inversa. El primero opera sobre bits mediante  $b_i = b_{i-r} \oplus b_{i-q}$  y agrupa  $L$  bits para formar cada número, de modo que su estado interno es la ventana de los últimos  $r$  bits y no el número de salida, distinción que importa para medir bien el periodo. El segundo aplica  $X_{n+1} = (aX_n^{-1} + c) \bmod m$  con  $m$  primo, calculando el inverso por el pequeño teorema de Fermat como  $X^{m-2} \bmod m$  y tomando por convención que el inverso de cero es cero; evita la estructura de retículo del congruencial a cambio de un costo por número bastante mayor.

```
1 bits.append(bits[-r] ^ bits[-q]) # shift-register
2
3 inverso = 0 if actual % modulo == 0 else pow(actual, modulo - 2, modulo)
4 actual = (multiplicador * inverso + incremento) % modulo # congruencia inversa
```

Listado 3: Recurrencias del shift-register y de la congruencia inversa.



## 4. Tamaño de ciclo y pruebas estadísticas

Cada secuencia se evaluó con  $n = 1000$  valores,  $\alpha = 0,05$  y  $k = 10$  clases. Se usó  $n = 1000$  en lugar de los 100 de la guía porque con diez clases la frecuencia esperada quedaría en el mínimo aceptable y el test de rachas apenas alcanzaría su aproximación normal.

### 4.1. Detección del periodo

Como los generadores son deterministas, apenas un estado se repite toda la secuencia se repite. Basta recorrer los estados guardando en qué posición apareció cada uno, y la primera repetición entrega tanto el transitorio como el ciclo.

```
1 def detectar_ciclo(estados):
2     posicion_de_estado = {}
3
4     for indice, estado in enumerate(estados):
5         if estado in posicion_de_estado:
6             inicio = posicion_de_estado[estado]
7             return inicio, indice - inicio
8         posicion_de_estado[estado] = indice
9
10    return None, None
```

Listado 4: Detección de transitorio y periodo.

### 4.2. Pruebas de uniformidad y aleatoriedad

La uniformidad se contrastó con  $\chi^2 = \sum (f_i - e_i)^2 / e_i$  sobre  $k$  clases de igual longitud con  $e_i = n/k$ , rechazando cuando supera  $\chi^2_{k-1, \alpha} = 16,92$ . La aleatoriedad se contrastó con el test de rachas, que construye una sucesión binaria según cada valor suba o baje respecto al anterior y compara el número de rachas  $R$  contra su distribución asintótica:

$$Z = \frac{R - \frac{2n-1}{3}}{\sqrt{\frac{16n-29}{90}}}, \quad \text{se rechaza si } |Z| > 1,96 \quad (4)$$

### 4.3. Resultados

Cuadro 1: Periodo y resultado de las pruebas para las dieciséis secuencias.

| Método           | Parámetros                | Periodo          | $\chi^2$ | Unif. | Z       | Aleat. |
|------------------|---------------------------|------------------|----------|-------|---------|--------|
| Cuadrados medios | semilla 1931              | 1                | 8960,06  | Falla | -49,945 | Falla  |
| Cuadrados medios | semilla 675248            | 1                | 5650,08  | Falla | -39,736 | Falla  |
| Cuadrados medios | semilla 4567              | 4                | 1455,44  | Falla | -12,712 | Falla  |
| Cuadrados medios | semilla 12122             | 1                | 7655,22  | Falla | -46,042 | Falla  |
| Cuadrados medios | semilla 7179345           | 20               | 10,70    | Pasa  | 0,651   | Pasa   |
| Congruencial     | $a=115, c=0, m=128$       | 32               | 54,74    | Falla | 1,476   | Pasa   |
| Congruencial     | $a=25, c=5, m=48$         | 48               | 6,84     | Pasa  | 3,053   | Falla  |
| Congruencial     | $a=16807, m=2^{31}-1$     | $> 3 \cdot 10^5$ | 22,84    | Falla | -1,001  | Pasa   |
| Congruencial     | $a=25214903917, m=2^{48}$ | $> 3 \cdot 10^5$ | 6,36     | Pasa  | 0,425   | Pasa   |
| Congruencial     | $a=5, c=7, m=200$         | 8                | 247,52   | Falla | 6,055   | Falla  |
| Shift-register   | $r=7, q=3, L=8$           | 127              | 2,38     | Pasa  | -1,601  | Pasa   |
| Shift-register   | $r=31, q=3, L=8$          | $> 3 \cdot 10^5$ | 12,26    | Pasa  | -0,400  | Pasa   |
| Shift-register   | $r=31, q=13, L=16$        | $> 3 \cdot 10^5$ | 5,46     | Pasa  | 1,777   | Pasa   |
| Cong. inversa    | $a=2, c=3, m=31$          | 31               | 10,02    | Pasa  | 7,932   | Falla  |
| Cong. inversa    | $a=5, c=11, m=101$        | 50               | 64,00    | Falla | 3,954   | Falla  |
| Cong. inversa    | $a=2, c=3, m=2147483053$  | $> 3 \cdot 10^5$ | 6,92     | Pasa  | -0,701  | Pasa   |

### 4.4. ¿Cuáles son los peores generadores y por qué?

El método de cuadrados medios queda último con diferencia. Cuatro de las cinco semillas llegan a un valor que al elevarse al cuadrado se devuelve a sí mismo y de ahí no salen: 1931 colapsa a los seis valores, 12122 aguanta 87 y 675248 llega a 209. El problema no es que produzca números poco aleatorios sino que deja de producir números, y por eso sus chi-cuadrado alcanzan 5650 y 8960 frente a un crítico de 16,92. La única semilla que sobrevive, 7179345, aprueba ambas pruebas, lo que resulta más inquietante que tranquilizador: un generador que depende de acertar con la semilla no ofrece garantía alguna.



**Figura 1:** Frecuencias por clase de cada secuencia. La línea roja marca la frecuencia esperada bajo uniformidad. Los títulos en verde corresponden a secuencias que aprueban el chi-cuadrado.

Siguen dos congruenciales mal parametrizados. Con  $a = 5$ ,  $c = 7$  y  $m = 200$  el periodo es de ocho valores, porque  $m = 2^3 \cdot 5^2$  y  $a - 1 = 4$  no es divisible por 5, lo que rompe la segunda condición de Hull-Dobell. Con  $a = 115$ ,  $c = 0$  y  $m = 128$  el periodo es 32, es decir  $m/4$ , que es el techo teórico de un multiplicativo con módulo potencia de dos. La congruencia inversa con  $m = 101$  cae en la misma categoría con periodo 50, lo que muestra que exigir módulo primo es necesario pero no suficiente. El patrón común es que en todos estos casos el periodo quedó por debajo de la cantidad de números solicitados, de modo que pedir mil valores a un generador de periodo ocho devuelve la misma lista repetida ciento veinticinco veces.

#### 4.5. ¿Cuáles son los mejores generadores y por qué?

Tres configuraciones aprueban ambas pruebas con periodos que ni siquiera pudieron medirse por fuerza bruta. El congruencial **drand48** ( $a = 25214903917$ ,  $c = 11$ ,  $m = 2^{48}$ ) obtiene  $\chi^2 = 6,36$  y  $Z = 0,425$  y cumple Hull-Dobell, por lo que su periodo teórico es  $2^{48}$ . El shift-register con  $r = 31$  y  $q = 13$  obtiene  $\chi^2 = 5,46$  y  $Z = 1,777$ , con periodo  $2^{31} - 1$  porque ese par corresponde a un trinomio primitivo. La congruencia inversa con módulo primo grande obtiene  $\chi^2 = 6,92$  y  $Z = -0,701$ ; es la más costosa por la exponenciación modular, pero evita la estructura de retículo del congruencial.

MINSTD merece una precisión: con la semilla 13 reprueba uniformidad ( $\chi^2 = 22,84$ ,  $p = 0,0066$ ) pero aprueba con otras cinco semillas, así que el resultado se explica mejor como ruido de muestreo que como defecto.

Lo más instructivo del cuadro son las dos secuencias que aprueban uniformidad y reprueban aleatoriedad, el congruencial con  $m = 48$  y la congruencia inversa con  $m = 31$ . Ambas tienen periodo completo, de modo que al pedirles mil números recorren su ciclo unas veintiuna y treinta y dos veces y cada valor aparece casi igual número de veces, dejando el histograma plano. Se trata sin embargo de la misma lista repetida en el mismo orden, y el test de rachas lo detecta de inmediato: un contador que recorre  $1, 2, \dots, 48$  en bucle es perfectamente uniforme y no tiene nada de aleatorio.

### 5. Uso de números aleatorios para evaluar integrales

El estimador de Monte Carlo se apoya en que la integral de  $g$  sobre  $[0, 1]$  coincide con el valor esperado de  $g(U)$  cuando  $U$  es uniforme, de modo que el promedio de  $g$  evaluada en  $n$  uniformes converge al valor buscado a razón de  $1/\sqrt{n}$ :

$$\int_0^1 g(x) dx = E[g(U)] \approx \frac{1}{n} \sum_{i=1}^n g(U_i) \quad (5)$$



Los ejercicios 5, 6, 7 y 9 no están definidos sobre  $[0, 1]$ , así que antes se transforma el dominio: cambio lineal  $x = a + (b - a)u$  con factor  $(b - a)$  para un intervalo finito,  $x = 1/u - 1$  con jacobiano  $1/u^2$  para un límite infinito, y doblado al lado positivo aprovechando la paridad del integrando en el ejercicio 7.

El ejercicio 9 requiere además la función indicadora sugerida en el enunciado: al escribir el límite interior como  $\int_0^\infty e^{-(x+y)} I(y < x) dy$  la región triangular se vuelve rectangular sin alterar el valor, y tras sustituir en ambos ejes la condición  $y < x$  se reduce a  $u < v$ .

```
1 def estimar_integral(funcion, generador, muestras, dimension=1):
2     valores = np.array([
3         funcion(*generador.siguientes(dimension)) for _ in range(muestras)
4     ])
5     return valores.mean(), valores.std(ddof=1) / np.sqrt(muestras)
6
7
8 def ejercicio_9(u, v):
9     """La indicadora  $I(y < x)$  se reduce a  $u < v$  tras la sustitucion."""
10    if u >= v:
11        return 0.0
12    x, y = 1 / u - 1, 1 / v - 1
13    return np.exp(-(x + y)) / (u ** 2 * v ** 2)
```

Listado 5: Estimador y ejemplo de función transformada.

Como fuente de uniformes se empleó el mejor generador de la sección anterior, el congruencial con  $m = 2^{48}$ , envuelto en una clase que descarta el valor cero porque las sustituciones dividen por  $u$ .

Cuadro 2: Estimación por Monte Carlo con  $n = 200000$  muestras.

| Ej. | Integral                                  | Monte Carlo | Referencia | Cerrada      |
|-----|-------------------------------------------|-------------|------------|--------------|
| 3   | $\int_0^1 e^x dx$                         | 6,303906    | 6,316564   | —            |
| 4   | $\int_0^1 (1 - x^2)^{3/2} dx$             | 0,588709    | 0,589049   | $3\pi/16$    |
| 5   | $\int_{-2}^2 e^{x+x^2} dx$                | 93,842542   | 93,162753  | —            |
| 6   | $\int_0^\infty x(1+x^2)^{-2} dx$          | 0,499139    | 0,500000   | $1/2$        |
| 7   | $\int_{-\infty}^\infty e^{-x^2} dx$       | 1,772826    | 1,772454   | $\sqrt{\pi}$ |
| 8   | $\int_0^1 \int_0^1 e^{(x+y)^2} dy dx$     | 4,891370    | 4,899159   | —            |
| 9   | $\int_0^\infty \int_0^x e^{-(x+y)} dy dx$ | 0,500238    | 0,500000   | $1/2$        |

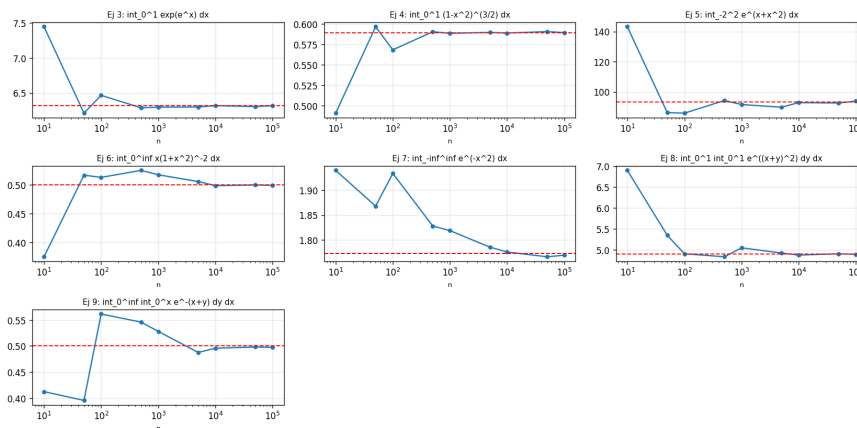


Figura 2: Convergencia del estimador frente al tamaño de muestra en escala logarítmica. La línea roja marca el valor de referencia.

Seis de los siete estimadores quedan dentro del 0,2 % del valor de referencia. El ejercicio 5 se aparta 0,73 %, equivalente a 1,2 errores estándar, lo que no indica un problema de la sustitución sino la varianza esperable de un integrando que alcanza  $e^6$  en el extremo derecho y vale casi nada en el centro.

## 6. Generación de variables aleatorias discretas

La transformada inversa acumula la función de masa y devuelve el primer valor cuya acumulada supera al uniforme generado. Los cuatro literales usan el congruencial mixto con  $X_0 = 2391$ ,  $a = 25214903917$ ,



$c = 11$  y  $m = 2^{48}$ , según el enunciado.

```
1 def transformada_inversa(u, valores, probabilidades):
2     acumulada = 0.0
3     for valor, probabilidad in zip(valores, probabilidades):
4         acumulada += probabilidad
5         if u < acumulada:
6             return valor
7     return valores[-1] # respaldo por error de redondeo
8
9
10 def generar_por_composicion(generator):
11     if generator.siguiente() < PESO_COMPONENTE_1: # 0.30
12         return int(generator.siguiente() * 5) + 1
13     return transformada_inversa(generator.siguiente(),
14                                VALORES_COMPONENTE_2, PROBABILIDADES_COMPONENTE_2)
```

**Listado 6:** Transformada inversa discreta y método de composición.

El literal (d) se resolvió por composición: las cinco primeras probabilidades son iguales a 0,06, lo que permite escribir la distribución como una mezcla donde con probabilidad 0,30 se sortea uniformemente entre 1 y 5 y con 0,70 se aplica transformada inversa sobre los valores del 6 al 10 reescalados:

$$P(X = j) = 0,30 \cdot P_1(j) + 0,70 \cdot P_2(j) \quad (6)$$

La ventaja práctica es que el treinta por ciento de las veces se evita recorrer la acumulada, reemplazándola por un índice calculado directamente.

**Cuadro 3:** Proporción de valores iguales a 1 en el literal (c), con  $p_1 = 1/3$  y  $p_2 = 2/3$ .

| $n$   | Proporción | Teórico | Error absoluto |
|-------|------------|---------|----------------|
| 100   | 0,3400     | 0,3333  | 0,0067         |
| 1000  | 0,3280     | 0,3333  | 0,0053         |
| 10000 | 0,3316     | 0,3333  | 0,0017         |

Con cien valores las frecuencias observadas se apartan visiblemente de las teóricas en los cuatro literales, así que se repitió el muestreo con cien mil para separar el error de muestreo de uno de implementación. La desviación máxima bajó a 0,00217 en (a), 0,00232 en (b), 0,00122 en (c) y 0,00116 en (d), confirmando que los cuatro generadores reproducen sus distribuciones objetivo.

## 7. Simulación ad hoc con variables aleatorias discretas

Se repitió la simulación de la cola de un servidor del Laboratorio 1 reemplazando los tiempos por variables generadas mediante transformada inversa: Poisson con  $\lambda = 10$  entre llegadas y Binomial con  $n = 10$  y  $p = 0,40$  para el servicio. Ambos generadores usan la recursión de la función de masa en lugar de evaluar factoriales, partiendo de  $p_0 = e^{-\lambda}$  y  $p_0 = (1 - p)^n$  respectivamente.

```
1 def generar_poisson(u, lam):
2     probabilidad = np.exp(-lam)
3     acumulada, i = probabilidad, 0
4     while u >= acumulada:
5         i += 1
6         probabilidad *= lam / i
7         acumulada += probabilidad
8     return i
9
10
11 def generar_binomial(u, ensayos, exito):
12     razon = exito / (1 - exito)
13     probabilidad = (1 - exito) ** ensayos
14     acumulada, i = probabilidad, 0
15     while u >= acumulada:
16         probabilidad *= (ensayos - i) / (i + 1) * razon
17         i += 1
18         acumulada += probabilidad
19     return i
20
21
22 for _ in range(clientes): # simulacion de la cola
23     reloj_llegada += generar_poisson(generator.siguiente(), lam)
24     inicio_servicio = max(reloj_llegada, fin_servicio)
```



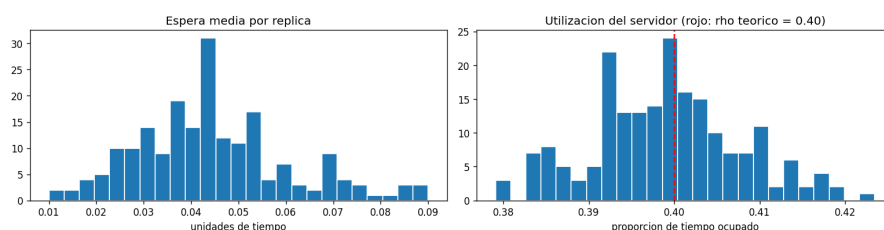
```
25 tiempo_ocioso += max(0.0, inicio_servicio - fin_servicio)
26 fin_servicio = inicio_servicio + generar_binomial(
27     generador.siguiente(), ensayos, exito)
28 esperas.append(inicio_servicio - reloj_llegada)
```

**Listado 7:** Generadores de Poisson y Binomial, y núcleo de la cola.

Sobre cincuenta mil valores el generador de Poisson arrojó media 9,9930 y varianza 9,9585 frente a los diez teóricos de ambos momentos, y el Binomial media 3,9856 y varianza 2,3945 frente a 4 y 2,4. Dado que cada corrida es una realización de un proceso aleatorio, los indicadores se acompañaron de su intervalo de confianza sobre doscientas réplicas independientes de quinientos clientes cada una.

**Cuadro 4:** Indicadores sobre 200 réplicas de 500 clientes.

| Indicador             | Media  | Desv.  | IC 95 %           |
|-----------------------|--------|--------|-------------------|
| Espera media          | 0,0444 | 0,0160 | [0,0422 ; 0,0466] |
| Espera máxima         | 3,6900 | 1,1793 | [3,5266 ; 3,8534] |
| Proporción sin espera | 0,9737 | 0,0080 | [0,9726 ; 0,9748] |
| Permanencia media     | 4,0366 | 0,0724 | [4,0266 ; 4,0466] |
| Servicio medio        | 3,9922 | 0,0671 | [3,9829 ; 4,0015] |
| Utilización           | 0,3991 | 0,0088 | [0,3979 ; 0,4003] |



**Figura 3:** Distribución de la espera media y de la utilización del servidor sobre las doscientas réplicas. La línea roja marca  $\rho = 0,40$ .

La utilización se estabiliza en 0,3991, que coincide con el cociente entre el servicio medio y el tiempo medio entre llegadas. El sistema queda holgado: cerca del 97 % de los clientes entra a servicio sin esperar y la espera media apenas supera 0,04 unidades de tiempo, de modo que la permanencia resulta prácticamente igual al tiempo de servicio.

La diferencia estructural frente al Laboratorio 1 es que allí los tiempos provenían de una tabla fija, lo que hacía la simulación reproducible y arrojaba un único valor por indicador, mientras que aquí cada corrida produce un resultado distinto que solo tiene sentido junto a su intervalo de confianza.

## 8. Conclusiones

El tamaño de ciclo resultó el criterio más discriminante: toda secuencia que reprobó alguna prueba tenía periodo inferior a la cantidad de números solicitados, y ninguna de las que superaba ese umbral falló por causa del generador. Uniformidad y aleatoriedad resultaron además propiedades independientes, pues dos generadores de periodo completo pero corto aprobaron el chi-cuadrado dejando el histograma plano mientras el test de rachas detectaba de inmediato la repetición del patrón.

La estimación de integrales mostró que la dificultad del método no está en el estimador sino en llevar el dominio a  $[0, 1]$ , y en la cola el paso de tiempos tabulados a tiempos generados convirtió cada indicador en una estimación que solo adquiere sentido junto a su intervalo de confianza.

## Referencias

- [1] Ross, S. *Simulación*, 2da Edición. Pearson Press, 1999.
- [2] Mancilla Herrera, A. M. Números aleatorios: historia, teoría y aplicaciones. *Ingeniería y Desarrollo*, núm. 8, pp. 49–69, Universidad del Norte, Barranquilla, 2000.